

## Table of contents

|                                                                                        |          |
|----------------------------------------------------------------------------------------|----------|
| <b>Series 11: Model Checking with Büchi Automata and LTL</b>                           | <b>1</b> |
| Methods . . . . .                                                                      | 1        |
| Construction of Büchi Automata (BA) from $\omega$ -Regular Expressions . . . . .       | 1        |
| Operations on Büchi Automata . . . . .                                                 | 1        |
| Translation between Properties, $\omega$ -Regular Expressions, LTL, and Büchi Automata | 2        |
| Satisfiability and Validity Verification for LTL Formulas . . . . .                    | 2        |
| Model Checking by Synchronous Product . . . . .                                        | 2        |
| Manipulation of Extended LTL Connectives . . . . .                                     | 3        |
| Reminder . . . . .                                                                     | 3        |
| Illustrative Examples . . . . .                                                        | 3        |
| Example 1: Translation of a Property into BA and $\omega$ -Regular Expression . . . .  | 3        |
| Example 2: Concatenation Operation FA and BA . . . . .                                 | 4        |
| Example 3: Model Checking by Synchronous Product . . . . .                             | 5        |
| Synthesis . . . . .                                                                    | 5        |

## Series 11: Model Checking with Büchi Automata and LTL

### Methods

#### Construction of Büchi Automata (BA) from $\omega$ -Regular Expressions

- **Objective:** Represent an infinite word language ( $\omega$ -language) described by an  $\omega$ -regular expression as a Büchi automaton.
- **Method:** Apply basic constructions for concatenation, union, Kleene star, and omega-power ( $\cdot^\omega$ ) operators, then combine them.

#### Operations on Büchi Automata

- **Concatenation of a Finite Automaton (FA) and a BA:**
  1. Keep states of both automata  $A$  (FA) and  $B$  (BA).
  2. The initial state is that of  $A$ .
  3. The accepting (final) states are those of  $B$ .
  4. Keep all existing transitions.
  5. For each transition in  $A$  going to one of its final states, add a similarly labeled transition from the origin to the initial state of  $B$ .
- **Omega-power of a Finite Automaton ( $A^\omega$ ):**
  1. Keep states of the finite automaton  $A$ .

2. The initial state is that of  $A$ .
3. The accepting states are those reachable from themselves via a cycle (i.e., part of an accessible strongly connected component).
4. All transitions are kept. The resulting automaton accepts infinite executions that visit at least one accepting state infinitely often.

### Translation between Properties, $\omega$ -Regular Expressions, LTL, and Büchi Automata

- **Objective:** Express a property described in natural language in different equivalent formal forms.
- **Process:**
  1. Understand the property's semantics (accepted vs rejected infinite words).
  2. Propose an  $\omega$ -regular expression that captures this semantics.
  3. Construct a BA that recognizes this expression.
  4. (If possible) Provide an equivalent LTL formula.

### Satisfiability and Validity Verification for LTL Formulas

- **Satisfiability:** Does there exist at least one infinite word (an execution) that satisfies the formula  $\phi$ ? Method: Propose a concrete counterexample  $w$  such that  $w \models \phi$ .
- **Validity:** Is the formula  $\phi$  true for all infinite words? Method: Prove validity by semantic reasoning, or show it is not valid by exhibiting a counterexample  $w$  such that  $w \not\models \phi$ .

### Model Checking by Synchronous Product

- **Objective:** Verify whether all behaviors (infinite words) of a system  $A$  satisfy a property  $\phi$  ( $A \models \phi$ ).
- **Method:**
  1. Negation: Construct the Büchi automaton  $B_{\neg\phi}$  for the negation of the property.
  2. Synchronous Product: Construct the product automaton  $A \times B_{\neg\phi}$  that recognizes the intersection of languages  $L(A) \cap L(\neg\phi)$ .
  3. SCC Analysis: Look in the product graph for a Strongly Connected Component (SCC) that is **accessible from the initial state** and **contains at least one accepting state** of  $B_{\neg\phi}$ .
    - If such an SCC exists:  $L(A) \cap L(\neg\phi) \neq \emptyset$ , so  $A \not\models \phi$ . A counterexample is a word in this SCC.
    - If no such SCC exists:  $L(A) \cap L(\neg\phi) = \emptyset$ , so  $A \models \phi$ .

## Manipulation of Extended LTL Connectives

- **Weak Until ( $p\mathbf{W}q$ ):** “ $p$  remains true until  $q$  becomes true, without guarantee that  $q$  arrives”. Formally:  $\sigma \models p\mathbf{W}q \iff (\sigma \models p\mathcal{U}q) \vee (\sigma \models \Box p)$ .
- **Infinitely Often ( $F^\infty p$ ):** “ $p$  is true infinitely often”. Formally:  $\sigma \models F^\infty p \iff \forall i \geq 0, \exists j \geq i, \sigma^j \models p$ . Can be expressed in standard LTL as  $\Box \diamond p$ .

## Reminder

- **Büchi Automaton:** Automaton on infinite words where a word is accepted if there exists an execution that visits **an accepting (final) state infinitely often**.
- **-Regular Expression:** Generalizes regular expressions to infinite languages. Basic operators are concatenation, union, Kleene star  $*$  (finite repetition), and the omega operator  $\omega$  (infinite repetition).
- **LTL Semantics:** An LTL formula is interpreted over an infinite word  $\sigma = \sigma_0\sigma_1\sigma_2\dots$ 
  - $\sigma \models \circ\phi \iff \sigma^1 \models \phi$
  - $\sigma \models \Box\phi \iff \forall i \geq 0, \sigma^i \models \phi$
  - $\sigma \models \diamond\phi \iff \exists i \geq 0, \sigma^i \models \phi$
  - $\sigma \models \phi\mathcal{U}\psi \iff \exists i \geq 0$  such that  $\sigma^i \models \psi$  and  $\forall 0 \leq j < i, \sigma^j \models \phi$
- **Synchronous Product:** Given  $A = (Q_A, \Sigma, \delta_A, q_{0A}, F_A)$  and  $B = (Q_B, \Sigma, \delta_B, q_{0B}, F_B)$ , the product  $A \times B$  has states  $Q_A \times Q_B$ , initial state  $(q_{0A}, q_{0B})$ , and a transition  $((s, t), a, (s', t'))$  if  $(s, a, s') \in \delta_A$  and  $(t, a, t') \in \delta_B$ . Accepting states depend on the automaton type.

## Illustrative Examples

### Example 1: Translation of a Property into BA and -Regular Expression

**Context:** Express the property “a response is eventually provided on each request”.

**Application:**

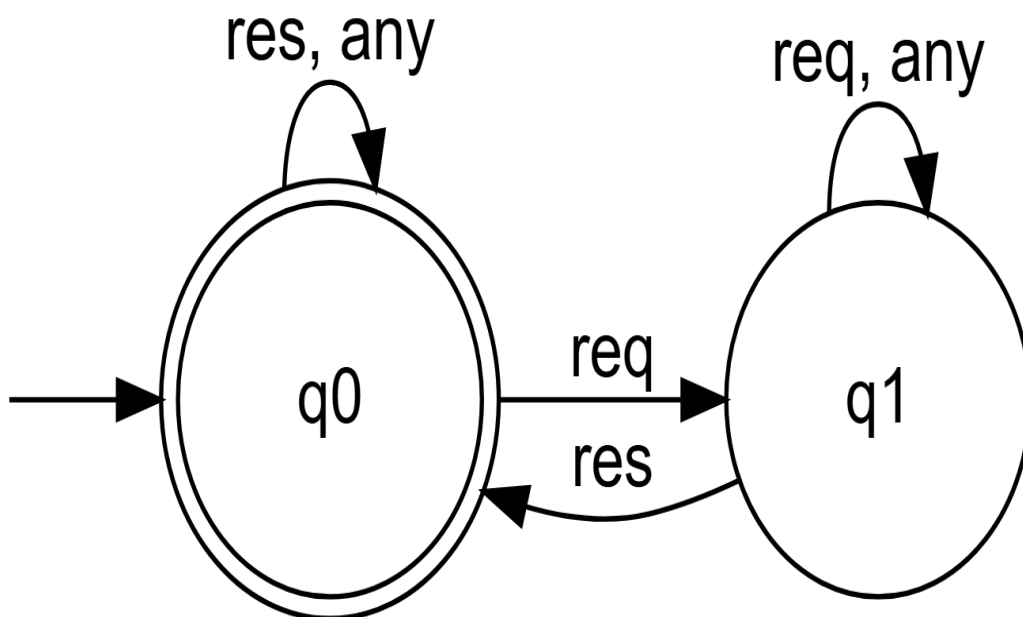
1. **Semantics:** After each occurrence of **req**, there must eventually be an occurrence of **res**. A word like  $(\mathbf{req} \text{ any}^* \mathbf{res})^\omega$  is accepted. A word like  $\mathbf{req} (\mathbf{any})^\omega$  (a request never served) is rejected.
2. **-Regular Expression:**

$$((\mathbf{req} \cup \mathbf{res} \cup \mathbf{any})^* \cdot \mathbf{res})^\omega \quad \cup \quad ((\mathbf{req} \cup \mathbf{res} \cup \mathbf{any})^* \cdot \mathbf{req} \cdot (\mathbf{any})^* \cdot \mathbf{res})^\omega$$

A simpler equivalent expression would be  $((\neg \mathbf{req})^* \cdot \mathbf{req} \cdot (\neg \mathbf{res})^* \cdot \mathbf{res})^\omega$

3. **Construction of the BA:** The automaton has two main states:

- $q_0$ : “Waiting for a request or nothing special”.
- $q_1$ : “A request has been issued and we await the response”.
- Transition from  $q_0$  to  $q_1$  on **req**.
- Transition from  $q_1$  to  $q_1$  on **any** except **res**.
- Transition from  $q_1$  to  $q_0$  on **res** (the request is served).
- All other transitions (e.g., **any** in  $q_0$ , **req** in  $q_1$ ) remain in the current state.
- The initial state is  $q_0$ . **All states are accepting** to ensure that pending requests (in  $q_1$ ) must be left (by going to  $q_0$  via **res**) for the execution to be accepted (because an accepting state must be visited infinitely often, so we cannot remain stuck in  $q_1$ ).



### Example 2: Concatenation Operation FA and BA

**Context:** Construct a BA for  $(ab+c)^*(ab^*)^\omega$  from an FA for  $(ab+c)^*$  and a BA for  $(ab^*)^\omega$ .

**Application:**

1. Let  $A$  be the FA for  $(ab+c)^*$ . It has an initial state  $i_A$  and (at least) a final state  $f_A$ .
2. Let  $B$  be the BA for  $(ab^*)^\omega$ . It has an initial state  $i_B$  and accepting states  $F_B$ .
3. **Construction of the BA for concatenation:**
  - States: Disjoint union of states of  $A$  and  $B$ .
  - Initial state:  $i_A$ .
  - Accepting states:  $F_B$  (only those of  $B$ ).
  - Transitions: All transitions of  $A$  and  $B$  are kept.

- **Added transitions:** For **each** transition in  $A$  of the form  $(p, \alpha, f_A)$  where  $f_A$  is a final state of  $A$ , add a **new** transition  $(p, \alpha, i_B)$  to the initial state of  $B$ .
4. **Justification:** To be accepted, a word must first be accepted by the finite prefix recognized by  $A$  (which leads to a final state  $f_A$ ), then immediately start and satisfy the infinite suffix recognized by  $B$ . The added transition enables this “handover”.

### Example 3: Model Checking by Synchronous Product

**Context:** Verify if the automaton  $A$  (with  $L(A) = (bc)^\omega$ ) satisfies  $\phi_2 = \Box(a \Rightarrow \Diamond c)$ .

**Application:**

1. **Negation of the property:**  $\neg\phi_2 = \neg\Box(a \Rightarrow \Diamond c) = \Diamond(a \wedge \Box\neg c)$ .
2. **Construction of  $B_{\neg\phi_2}$ :**
  - It seeks to capture a moment where  $a$  is true and then  $c$  is **always false**.
  - The shown automaton has states representing “has not yet seen  $a \wedge \Box\neg c$ ” (initial), “has seen  $a$  and now verifies  $\Box\neg c$ ” (accepting).
  - It remains in the accepting state only when reading letters where  $c$  is false.
3. **Synchronous Product  $A \times B_{\neg\phi_2}$ :**
  - We couple states of  $A$  (e.g.,  $s_1, s_2, s_3$ ) with those of  $B$ .
  - Automaton  $A$  only generates letters **b** and **c**. It **never** generates the letter **a**.
4. **Analysis:** The premise  $a$  of the formula  $\phi_2$  is never true in any word of  $L(A)$ . Therefore, the implication  $a \Rightarrow \Diamond c$  is **vacuously true** at every position. The formula  $\phi_2$  is thus true on every word of  $L(A)$ .
5. **Verification by SCC:** In the product, no execution can reach the accepting state of  $B_{\neg\phi_2}$  because that requires reading an **a**. Hence, no accessible SCC contains an accepting state of  $B_{\neg\phi_2}$ . We conclude that  $L(A) \cap L(\neg\phi_2) = \emptyset$ , so  $A \models \phi_2$ .

### Synthesis

This series of exercises covers the core of automata-based model checking. The key skills are:

1. formal manipulation of  $\omega$ -languages via expressions and Büchi automata
2. translation between different notations (LTL, expressions, automata)
3. use of automata operations to construct complex representations
4. application of the central model checking algorithm via synchronous product and SCC search.

A deep understanding of LTL semantics and Büchi acceptance is fundamental for all these tasks.